E4 Educator v2.0

T01

PlatformIO and the E4 in depth

Tier 1 · Tutorial 1 of 7

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How PlatformIO and the ESP32 toolchain fit together
- Why we pin the platform version and use build_flags
- Why delay() is a problem and how millis() solves it
- How to use Serial for debugging — the right way
- Fundrum firmware conventions you'll use in every project

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- VS Code + PlatformIO installed (see Quick Start Guide)
- USB-C cable (data-capable)

1 How it all fits together

Before writing code, it's worth understanding what's actually happening when you press upload. This mental model will save you hours of confusion later.

1.1 The stack

When you write ESP32 firmware in PlatformIO, you're working with several layers stacked on top of each other:

Layer	What it is
Your code	The firmware you write — setup(), loop(), your functions
Arduino framework	The friendly wrapper — Serial, pinMode, digitalWrite, Wire, SPI. Makes the ESP32 approachable.
ESP-IDF	Espressif's own SDK running underneath Arduino. FreeRTOS lives here. You rarely touch this directly but it's always there.
ESP32 hardware	The physical chip — dual core Xtensa LX6, 240MHz, WiFi, Bluetooth, and a rich set of peripherals.

Key point
Arduino on ESP32 is a convenience wrapper over the ESP-IDF. The friendlier it looks, the more hidden complexity lurks underneath. Understanding this helps you make sense of behaviour that would otherwise seem mysterious.

1.2 What PlatformIO does

PlatformIO is the glue that holds everything together. When you press upload it:

- Reads your platformio.ini to know which board, framework, and platform version to use
- Downloads the correct toolchain if it's not already installed
- Compiles your code using the Xtensa GCC compiler
- Packages the binary into an ESP32 flash image
- Uploads it to the ESP32 over USB via the CP2102 serial chip
- Resets the board so your new firmware starts running

All of this happens in seconds. The platformio.ini file is the key that controls the whole process.

2 Understanding platformio.ini

The platformio.ini file is the most important file in your project. Every E4 project starts with the same base configuration — understanding each line means you'll never be confused by it.

```
[env:e4_educator_v2]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed = 115200
monitor_port  = COM10
upload_port   = COM10
```

```
build_flags =
  -DFW_VERSION=\"1.0.0\"
  -DFW_DATE=\"2026-05-01\"
  -DLED_RED=16 -DLED_GREEN=26 -DLED_BLUE=32
  -DBTN_NEXT=13 -DBTN_ENT=14
  ; ... all other pin definitions
```

2.1 Line by line

Line	What it means
`platform = espressif32@6.6.0`	Pins the ESP32 toolchain to version 6.6.0. Never float this — floating versions cause silent breaking changes.
`board = esp32dev`	Tells PlatformIO the target is an ESP32 Dev Module. Sets flash size, CPU frequency, and partition defaults.
`framework = arduino`	Use the Arduino framework on top of ESP-IDF. Gives us setup(), loop(), Serial, Wire, SPI etc.
`monitor_speed = 115200`	Serial monitor baud rate. Must match Serial.begin(115200) in your code. Always 115200 on E4 projects.
`monitor_port = COM10`	Your PC's COM port for the E4 board. Change this to match your system — check Device Manager.
`build_flags = -DLED_RED=16`	Passes a #define to the compiler. -DLED_RED=16 is equivalent to writing #define LED_RED 16 at the top of every source file.

Key point

The -D prefix in build_flags means "define". Every GPIO pin on the E4 is defined here and nowhere else. Your firmware code uses names like LED_RED and BTN_NEXT — never raw numbers. If a pin ever changes, you update platformio.ini in one place and everything else automatically follows.

3 The delay() problem — and how to fix it

This is one of the most important things you'll learn in this course. The Quick Start Guide used delay() to blink the LEDs. It worked — but it introduced a serious limitation.

3.1 What delay() actually does

When you call delay(500), you're telling the ESP32 to do absolutely nothing for 500 milliseconds. Not check buttons. Not read sensors. Not handle WiFi. Not update a display. Nothing.

For a simple blink this is fine. But as soon as your firmware needs to do more than one thing — read a button while blinking an LED, for example — delay() becomes a wall that blocks everything else.

```
// This looks fine but...
void loop() {
  digitalWrite(LED_RED, HIGH);
  delay(500);               // Nothing else can happen here
  digitalWrite(LED_RED, LOW);
  delay(500);               // Or here
                            // Button presses during these 500ms are missed
}
```

3.2 The millis() solution

millis() returns the number of milliseconds since the board was powered on. Instead of blocking and waiting, we check whether enough time has passed and act accordingly. The loop() keeps running freely — checking everything on every pass.

```
#include <Arduino.h>

// Timing variables — one per timed event
unsigned long lastLedToggle = 0;
const unsigned long LED_INTERVAL = 500;  // ms

bool ledState = false;

void setup() {
  Serial.begin(115200);
  pinMode(LED_RED, OUTPUT);
  pinMode(BTN_NEXT, INPUT_PULLUP);
}

void loop() {
  unsigned long now = millis();

  // Blink the LED every 500ms — non-blocking
  if (now - lastLedToggle >= LED_INTERVAL) {
    lastLedToggle = now;
    ledState = !ledState;
    digitalWrite(LED_RED, ledState);
  }
```

```
  // Read the button — this runs on EVERY loop pass
  if (digitalRead(BTN_NEXT) == LOW) {
    Serial.println("NEXT pressed!");
  }
}
```

Notice that the button check runs on every single pass through loop() — even while the LED is mid-blink. Nothing is blocked. This is the pattern used in every Fundrum project.

Key point

The millis() pattern is: store the time of the last event, check if enough time has passed, act and update the stored time. Every timed event gets its own lastXxx variable and interval constant. This is the foundation of all non-blocking ESP32 firmware.

4 Serial debugging — your best tool

The Serial monitor is your window into what's happening inside the ESP32. Used well, it makes debugging fast and satisfying. Used poorly, it causes its own problems.

4.1 Basic Serial use

```
void setup() {
  Serial.begin(115200);
  Serial.println("Firmware started");
  Serial.print("Version: ");
  Serial.println(FW_VERSION);   // From build_flags
}

void loop() {
  int reading = analogRead(LDR);
  Serial.print("LDR: ");
  Serial.println(reading);
  // Note: no delay here in real code — use millis()
}
```

4.2 Serial printf — cleaner output

For formatted output, Serial.printf() is much cleaner than multiple print calls:

```
// Instead of this:
Serial.print("Temp: ");
Serial.print(temperature);
Serial.print(" C  Humidity: ");
Serial.println(humidity);

// Use this:
Serial.printf("Temp: %.1f C  Humidity: %.1f%%\n", temperature, humidity);
```

4.3 Serial and GPIO 1/3

> ⚠ **Important**
> GPIO 1 is UART0 TX and GPIO 3 is UART0 RX — the Serial monitor pins. On the E4 v2.0 these are broken out as a UART header for external use, not connected to LEDs. Never use GPIO 1 or 3 for output in your firmware — they are reserved for Serial.

4.4 Opening the Serial monitor

1. Make sure your firmware has Serial.begin(115200) in setup()
2. Upload your firmware
3. Press Ctrl+Alt+S in VS Code to open the Serial monitor
4. Confirm the baud rate shows 115200 in the bottom toolbar
5. Press the RST button on the E4 board to see startup messages

5 Fundrum firmware conventions

Every project in the E4 Educator course follows the same conventions. These aren't arbitrary rules — each one exists because it prevents a real problem.

Convention	Rule	Why
Platform version	`Always espressif32@6.6.0`	Floating versions cause silent breaking changes
GPIO numbers	Always in build_flags, never in code	One change point, consistent across all files
Timing	`millis() always, delay() never`	Non-blocking firmware is responsive firmware
Serial speed	Always 115200 baud	Consistent across all projects and team members
Versioning	`FW_VERSION and FW_DATE in build_flags`	Every build is traceable and identifiable
NVS keys	Maximum 15 characters	NVS silently fails with longer keys
ADC pins	ADC1 only when WiFi is active	ADC2 conflicts with the WiFi radio

6 Practical exercise

Let's put everything together. This exercise builds a non-blocking multi-LED blinker that also reads the NEXT button and reports everything to Serial.

6.1 Create the project

6. Create a new PlatformIO project called E4_T01_Exercise
7. Copy the full platformio.ini from the Quick Start Guide, updating your COM port
8. Open src/main.cpp and enter the code below

```
#include <Arduino.h>

// ── Timing ──────────────────────────────────────────────────
unsigned long lastRed   = 0;
unsigned long lastGreen = 0;
unsigned long lastBlue  = 0;
unsigned long lastReport = 0;

const unsigned long RED_INTERVAL   = 300;
const unsigned long GREEN_INTERVAL = 600;
const unsigned long BLUE_INTERVAL  = 900;
const unsigned long REPORT_INTERVAL = 2000;

// ── State ───────────────────────────────────────────────────
bool redState   = false;
bool greenState = false;
bool blueState  = false;
int  buttonCount = 0;
```

```
void setup() {
  Serial.begin(115200);
  Serial.printf("E4 Educator v2.0  FW: %s  Date: %s\n",
                FW_VERSION, FW_DATE);

  pinMode(LED_RED,   OUTPUT);
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_BLUE,  OUTPUT);
  pinMode(BTN_NEXT,  INPUT_PULLUP);

  digitalWrite(LED_RED,   LOW);
  digitalWrite(LED_GREEN, LOW);
  digitalWrite(LED_BLUE,  LOW);

  Serial.println("Ready. Press NEXT button.");
}

void loop() {
  unsigned long now = millis();

  // Blink RED every 300ms
  if (now - lastRed >= RED_INTERVAL) {
    lastRed = now;
    redState = !redState;
    digitalWrite(LED_RED, redState);
  }

  // Blink GREEN every 600ms
  if (now - lastGreen >= GREEN_INTERVAL) {
    lastGreen = now;
    greenState = !greenState;
    digitalWrite(LED_GREEN, greenState);
  }

  // Blink BLUE every 900ms
  if (now - lastBlue >= BLUE_INTERVAL) {
    lastBlue = now;
    blueState = !blueState;
    digitalWrite(LED_BLUE, blueState);
  }

  // Count NEXT button presses (LOW = pressed, INPUT_PULLUP)
  if (digitalRead(BTN_NEXT) == LOW) {
    buttonCount++;
    Serial.printf("NEXT pressed! Count: %d\n", buttonCount);
    delay(200);  // Simple debounce — we improve this in T03
  }

  // Report status every 2 seconds
  if (now - lastReport >= REPORT_INTERVAL) {
    lastReport = now;
    Serial.printf("[%lu ms] R:%d G:%d B:%d  BTN:%d\n",
```

```
                    now, redState, greenState, blueState, buttonCount);
  }
}
```

6.2 What to observe

- All three LEDs blink at different rates simultaneously — none of them block the others
- Pressing NEXT is registered immediately, even mid-blink — nothing is blocked
- The Serial monitor shows a status report every 2 seconds without interrupting the blinking
- The firmware version and date print at startup — from build_flags, not hardcoded

Try this
Change RED_INTERVAL to 100 and BLUE_INTERVAL to 1500. Upload again. Notice how the LEDs immediately adopt the new timing — you only changed two numbers in one place. This is why the interval constants matter.

7 What's next

You've covered a lot of ground in T01. The mental models you've built here — the stack, build_flags, millis() timing, Serial debugging — underpin every tutorial that follows.

- T02 — GPIO and build_flags in depth: understanding INPUT_PULLUP, output modes, and the full pin convention
- T03 — Buttons and debouncing: replacing the delay(200) bodge with a proper debounce implementation
- T04 — I2C fundamentals: reading your first sensor — the AHT20 temperature and humidity module already on the E4

8 T01 summary

Concept	Key takeaway
The stack	Your code → Arduino → ESP-IDF → hardware. Each layer adds convenience and hides complexity.
platformio.ini	Controls everything. Pin the platform version. Never float it.
build_flags	All GPIO numbers live here and nowhere else. -DLED_RED=16 becomes #define LED_RED 16 everywhere.
delay()	Blocks everything. Fine for demos, not for real firmware.
millis()	Non-blocking timing. Store last event time, check elapsed, act and update. One variable per timed event.
Serial.printf()	Cleaner than multiple print calls. Always 115200 baud. Ctrl+Alt+S to open monitor.
ADC1 rule	When WiFi is active, only use ADC1 pins (GPIO 32–39) for analog reads.